TỔNG KẾT ĐỀ TÀI

Phân loại ảnh MRI não bằng Tiny CNN INT8

Tăng tốc suy luận trên SoC PYNQ-Z2 bằng ARM + FPGA

Slide 08 | Kiến thức tổng hợp – kết quả – thảo luận

Mục tiêu của bài tổng kết

– Liên kết sáu Lab thành một quy trình triển khai hệ thống hoàn chỉnh.

– Giải thích vì sao chỉ đưa **Conv1 + ReLU + MaxPool1** lên FPGA.

– Đọc đúng số liệu **Accuracy, Macro F1 và Speedup**.

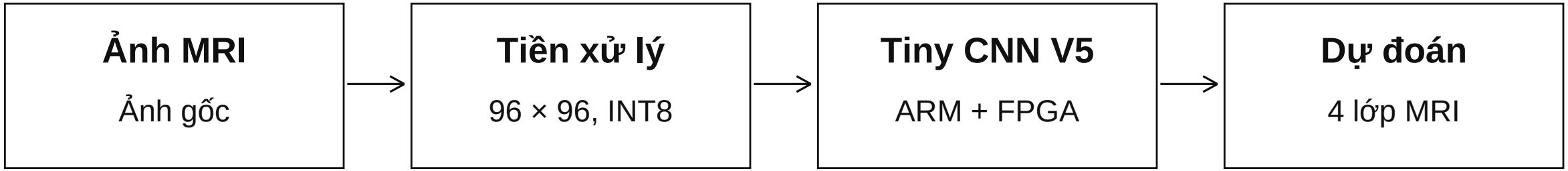
– Biết kết luận điều gì từ thực nghiệm, và điều gì chưa thể kết luận.

Từ Lab 01 đến Lab 06: một câu chuyện kỹ thuật

Lab	Câu hỏi chính	Sản phẩm
01	Board đã sẵn sàng chưa?	PYNQ, Python, Overlay hoạt động
02	Dữ liệu có sạch không?	5.950 ảnh sau làm sạch
03	CNN nào được chọn?	Tiny CNN V5 INT8
04	Đưa Conv1 lên FPGA ra sao?	IP HLS và bitstream Vivado
05	Hai phần phối hợp thế nào?	Suy luận Hybrid trên board
06	Hệ thống tốt hơn bao nhiêu?	Đánh giá 995 ảnh MRI

Hệ thống làm việc như thế nào?

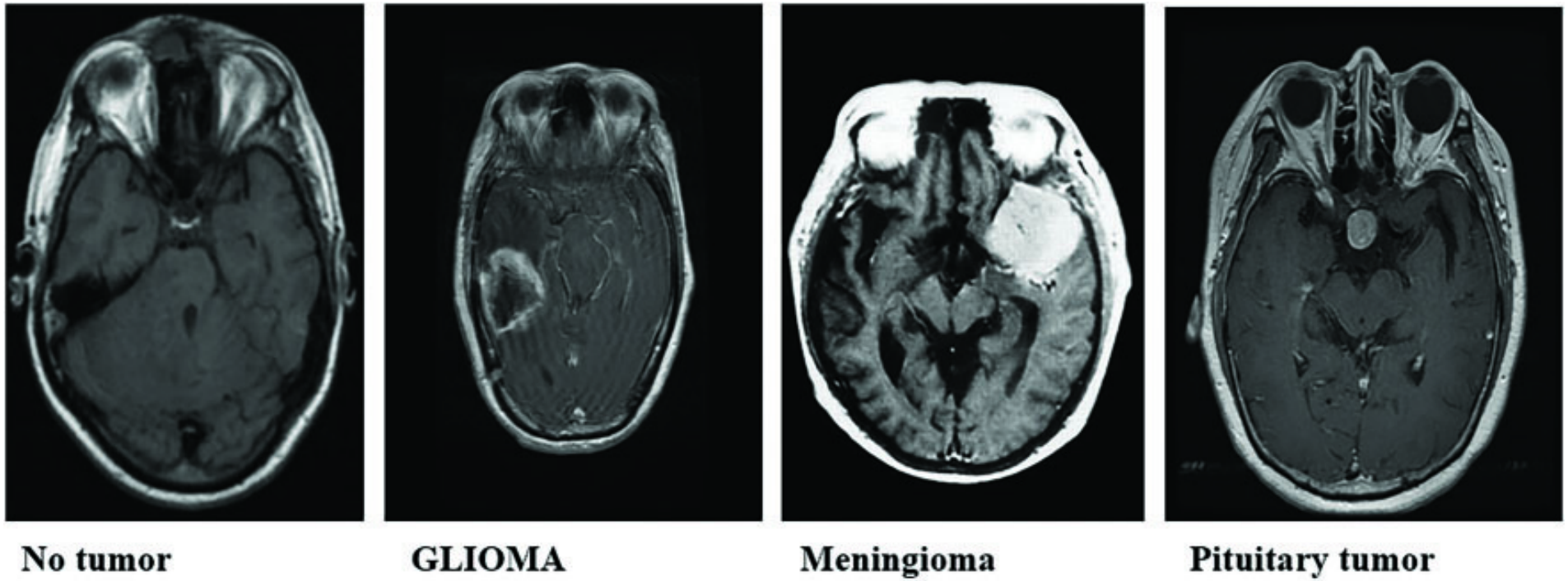
Hệ thống nhận một ảnh MRI và tạo ra một trong bốn nhãn phân loại.



Điểm quan trọng: FPGA không thay thế ARM, mà tăng tốc một đoạn tính toán của CNN.

Dữ liệu MRI: bốn lớp cần phân loại

Các mẫu MRI minh họa các dạng ảnh đầu vào của bài toán phân loại.



No tumor – Glioma – Meningioma – Pituitary

Hình minh họa từ tài liệu nghiên cứu, không đại diện toàn bộ tập kiểm thử.

Vì sao dữ liệu phải được kiểm tra trước?

– Bộ dữ liệu gốc: **6.000 ảnh**, gồm 5.000 Train và 1.000 Test.

– Kiểm tra SHA-256 phát hiện 50 tệp trùng dư thừa; trong đó có các trường hợp trùng giữa Train và Test.

– Sau làm sạch: **4.955 Train + 995 Test = 5.950 ảnh**.

– Kiểm tra SHA-256 chỉ loại được tệp giống hệt byte, không loại được ảnh gần trùng hay trùng bệnh nhân.

Train, Validation và Test khác nhau thế nào?

Tập	Số ảnh	Vai trò
Train	3.964	Cập nhật trọng số CNN
Validation	991	Chọn mô hình và dừng sớm
Test	995	Đánh giá sau khi chốt mô hình

Không dùng tập Test để điều chỉnh mô hình; nếu không, kết quả cuối có nguy cơ lạc quan.

Tiny CNN V5: những thành phần cốt lõi

Thành phần	Cấu hình	Kết quả
Input	Ảnh xám INT8	96 × 96 × 1
Conv1 + ReLU + Pool1	3 × 3, 8 kênh, valid	47 × 47 × 8
Conv2 / Conv3 / Conv4	24 / 48 / 64 kênh	Đặc trưng mức cao
Classifier	Flatten, Dense 64, Softmax	4 điểm số

Điểm phân chia phần cứng được đặt ngay sau MaxPool1.

Phân chia công việc giữa ARM và FPGA

ARM / Processing System

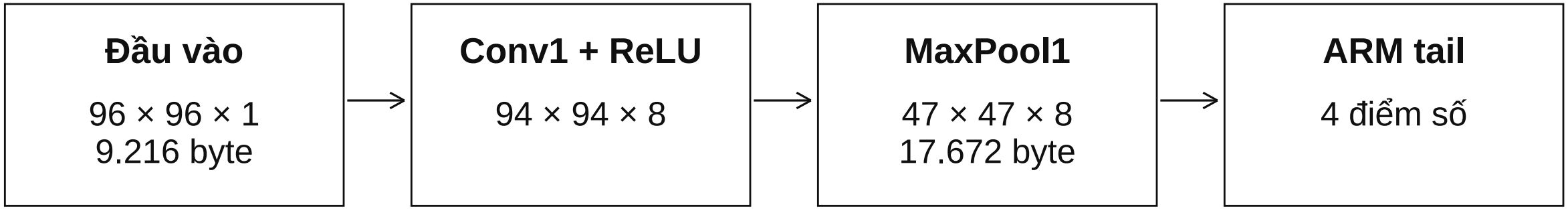
– Chuẩn bị dữ liệu ảnh
– Điều khiển DMA và IP
– Chạy phần mạng còn lại

FPGA / Programmable Logic

– Tính Conv1
– Kích hoạt ReLU
– MaxPooling 2 × 2

Việc phân chia nhằm khai thác phép toán lặp, song song của Conv1 mà vẫn giữ khả năng điều khiển linh hoạt của ARM.

Kích thước dữ liệu qua khối tăng tốc



Tensor sau MaxPool1 là giao diện dữ liệu chung giữa IP trên FPGA và mô hình tail trên ARM.

Vì sao chọn INT8 thay vì Float32?

– **Float32** biểu diễn dữ liệu bằng số thực 32 bit; **INT8** dùng số nguyên có dấu 8 bit.

– INT8 giúp giảm dung lượng biểu diễn và phù hợp với cấu trúc tính toán số nguyên trên FPGA.

– Lượng tử hóa không chỉ là ép kiểu: phải sử dụng **scale** và **zero-point** phù hợp.

– Đầu ra FPGA phải khớp chuẩn số học của mô hình TFLite, không chỉ cho cùng nhãn.

Từ phép tích chập đến khối phần cứng

Conv1 dùng 8 bộ lọc 3 × 3 trên ảnh xám đầu vào.

Số phép MAC của Conv1

$$94 \times 94 \times 8 \times 3 \times 3 = 636.192 \text{ MAC}$$

MAC (Multiply–Accumulate): một phép nhân cộng tích lũy.

Các phép tính lặp ở các vị trí ảnh và kênh đầu ra tạo cơ hội để thiết kế pipeline và xử lý song song.

AXI DMA đóng vai trò gì?

– Ảnh đầu vào INT8 được đặt trong bộ nhớ DDR.

– **DMA** truyền dữ liệu vào IP Conv1 mà không phải để ARM chép từng byte trong vòng lặp phần mềm.

– Khi FPGA hoàn thành Pool1, DMA chuyển tensor kết quả về DDR.

– ARM nạp tensor này vào **TFLite tail** và tính bốn điểm số phân loại.

Kiểm chứng kết quả: đúng trước, nhanh sau

Cả hai đường thực thi phải cho cùng tensor phân loại INT8.

ARM thuần

Tiny CNN V5 INT8 đầy đủ

ARM + FPGA

Conv1 trên PL → ARM tail

995/995 tensor đầu ra trùng khớp

Điều này bảo đảm phép tăng tốc không làm thay đổi kết quả của hai đường chạy trong bộ kiểm thử đã thực hiện.

Đo thời gian nào, không đo thời gian nào?

Có tính	Không tính
Thực thi mạng TFLite trên ARM	Đọc file ảnh từ bộ nhớ
Truyền dữ liệu DMA và gọi IP	Resize và chuyển đổi ảnh
Chạy tail sau FPGA	Nạp Overlay / khởi tạo Interpreter

Vì vậy, thời gian công bố là **độ trễ suy luận** theo phạm vi đã định nghĩa, không phải tổng thời gian ứng dụng từ đầu đến cuối.

Ba kết quả quan trọng nhất

92,66%

Accuracy trên 995 ảnh

0,9194

Macro F1

995/995

Tensor INT8 khớp

922 ảnh phân loại đúng, 73 ảnh phân loại sai.

Accuracy là tỷ lệ dự đoán đúng; Macro F1 cân bằng đánh giá giữa các lớp.

Phân tích ma trận nhầm lẫn

Hàng là nhãn thật, cột là nhãn dự đoán (số ảnh).

Thật / Dự đoán	Glioma	Mening.	No tumor	Pituitary
Glioma	244	7	0	2
Meningioma	17	264	12	10
No tumor	3	13	117	6
Pituitary	1	2	0	297

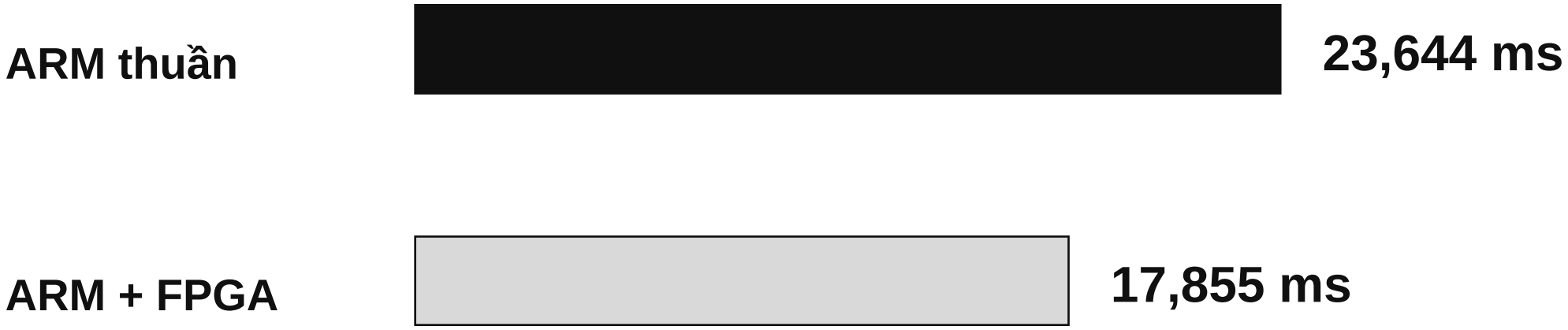
Sai số đáng chú ý ở Meningioma (39 ảnh sai) và No tumor (22 ảnh sai).

Vì sao không chỉ nhìn Accuracy?

Lớp	Recall	Số ảnh đúng / tổng
Glioma	96,44%	244 / 253
Meningioma	87,13%	264 / 303
No tumor	84,17%	117 / 139
Pituitary	99,00%	297 / 300

Kết luận: chất lượng phân loại giữa các lớp chưa đồng đều; No tumor có Recall thấp nhất.

Hiệu năng: ARM thuần và ARM + FPGA



Trung vị thời gian suy luận trên 995 ảnh; không gồm đọc ảnh, resize và khởi tạo.

Tăng tốc 1,324× có ý nghĩa gì?

$$\text{Speedup} = T_{ARM} / T_{Hybrid}$$

$$23{,}644 / 17{,}855 \approx 1{,}324 \text{ lần}$$

– Giảm trung vị khoảng **5,789 ms** cho mỗi ảnh.

– Tương đương giảm khoảng **24,48%** thời gian suy luận.

– Đây không phải tăng Accuracy; hai cấu hình có cùng dự đoán.

Không nhầm kết quả Lab 05 và Lab 06

Phép đánh giá	ARM thuần	ARM + FPGA	Speedup
Lab 05: một ảnh, 30 lượt	23,626 ms	18,055 ms	1,309×
Lab 06: 995 ảnh Test	23,644 ms	17,855 ms	1,324×

Báo cáo tổng kết lấy kết quả trên **995 ảnh Test** làm kết quả chính.

Vì sao FPGA không nhanh hơn hàng chục lần?

– FPGA chỉ tăng tốc phần đầu **Conv1 + ReLU + Pool1**, không phải toàn bộ CNN.

– Dữ liệu vẫn đi qua **DMA** và các bước đồng bộ bộ nhớ.

– Phần còn lại của mạng vẫn chạy trên ARM và chiếm phần lớn độ trễ.

– Ước lượng thời gian tính toán riêng trong HLS không tương đương thời gian đo cả hệ thống.

Giới hạn nghiên cứu phải trình bày rõ

– Khác biệt tiền xử lý giữa **Edge Impulse** và **PYNQ-Z2** chưa được xác minh khớp từng pixel.

– Dữ liệu chưa được xác nhận chia độc lập theo từng bệnh nhân.

– Thực nghiệm chưa đo công suất hoặc năng lượng tiêu thụ.

– Phép đo 995 ảnh là một lượt đánh giá chính, chưa có khoảng tin cậy thống kê.

Nếu phát triển tiếp, ưu tiên những việc gì?

– Chuẩn hóa tiền xử lý để Edge Impulse và PYNQ-Z2 tạo cùng tensor đầu vào.

– Kiểm tra trực tiếp tensor sau Pool1, không chỉ tensor đầu ra phân loại.

– Đánh giá trên tập ảnh tách độc lập theo bệnh nhân và đo lặp nhiều lượt.

– Khảo sát tăng tốc thêm các lớp CNN; đo riêng chi phí DMA và năng lượng.

Gợi ý trình tự video thuyết trình tổng kết

Phần trình bày	Nên nói điều gì
Mở đầu	Bài toán, dữ liệu và mục tiêu tăng tốc
Cơ sở lý thuyết	CNN, lượng tử hóa INT8, ARM/FPGA
Cách triển khai	6 Lab, ranh giới Conv1 – tail, AXI DMA
Thực nghiệm	So sánh kết quả 995 ảnh và thời gian
Kết luận	Đóng góp, giới hạn và hướng phát triển

Lời kết gợi ý cho video

“Nhóm đã xây dựng một hệ thống phân loại bốn nhóm ảnh MRI não bằng mô hình Tiny CNN V5 INT8 trên PYNQ-Z2. Điểm chính của nghiên cứu là đưa khối Conv1, ReLU và MaxPool1 sang FPGA, trong khi ARM tiếp tục thực thi phần mạng còn lại.”

“Trên 995 ảnh kiểm thử, hệ thống đạt Accuracy 92,66%, Macro F1 là 0,9194 và hai phương án cho tensor đầu ra INT8 giống nhau. Thời gian suy luận trung vị giảm từ 23,644 ms xuống 17,855 ms, tương ứng tăng tốc 1,324 lần.”

Câu hỏi ôn tập khi đứng lớp

– Vì sao nhóm chỉ tăng tốc Conv1 mà không đưa toàn bộ mạng lên FPGA?

– Tại sao hai phương án phải khớp toàn bộ tensor INT8 trước khi so thời gian?

– Accuracy 92,66% khác gì với Macro F1 0,9194?

– Vì sao không được khẳng định hệ thống tăng 1,324× ở mọi ứng dụng thực tế?

– Bạn sẽ ưu tiên thay đổi điều gì nếu tiếp tục nghiên cứu?

KẾT LUẬN

CNN INT8 + ARM + FPGA trên PYNQ-Z2

92,66% Accuracy | 0,9194 Macro F1 | 1,324× Speedup

Giữ nguyên kết quả phân loại trong thử nghiệm, đồng thời giảm thời gian suy luận.